

# LangChain4j: A Comprehensive Guide

## 1. Introduction to LangChain4j

LangChain4j is a Java framework designed to simplify the integration of Large Language Models (LLMs) into Java applications. It provides a unified API, allowing developers to switch between different LLM providers like OpenAI, Google Gemini, Anthropic, and Local models without rewriting their core business logic.

The goal of LangChain4j is to bring the power of AI to the Java ecosystem, which is traditionally used for enterprise backend systems. By offering abstractions for concepts like Chat Models, Embedding Models, Vector Stores, and Document Splitters, it significantly reduces the boilerplate required to build modern AI-powered applications.

## 2. Core Concepts

### 2.1 Chat Language Models

At the heart of any conversational AI application is the Chat Language Model. LangChain4j abstracts this via the `ChatLanguageModel` interface. Developers can easily configure an instance by providing API keys and model names. For example, switching from `gpt-4o` to `gemini-1.5-pro` is as simple as changing the builder implementation.

### 2.2 Vector Stores and Embeddings

To give LLMs context about specific data (like company policies or personal documents), we use Vector Stores. Text is first converted into numerical representations called Embeddings using an `EmbeddingModel`. These arrays of numbers are then stored in a `VectorStore`.

When a user asks a question, their query is also embedded, and the system searches the Vector Store for the most similar embeddings representing relevant text chunks. LangChain4j supports many stores, including Pinecone, Weaviate, Chroma, and a simple `InMemoryEmbeddingStore` for testing.

### 2.3 Document Parsing and Chunking

LLMs have a context window limit (the maximum amount of text they can process at once). If you want to feed a 500-page book to an LLM, you must split it. LangChain4j provides `DocumentSplitters` for this purpose.

You can split documents: recursively (keeps paragraphs together), byParagraph, bySentence, or byCharacter (strict). LangChain4j also integrates with Apache Tika to automatically extract text from various file formats, including PDF, DOCX, and PPTX.

### **3. Building a RAG Pipeline**

A typical pipeline involves loading documents (FileSystemDocumentLoader), parsing the files into text (ApacheTikaDocumentParser), splitting the text (DocumentSplitter), generating embeddings (EmbeddingModel), storing in a database (EmbeddingStore), and retrieving chunks to pass to the LLM (AiServices).